

# سازمان نرم‌افزاری مجازی

یک بسته آماده استفاده که یک «تیم توسعه نرم‌افزار» از عامل‌های هوش مصنوعی را برای شما تعریف می‌کند – همراه با قوانین، تحویل کار، کنترل کیفیت و نقاط تأیید انسانی.

آماده برای Git

• فایل زائد

• فایل تکراری

۱۹۶ فایل تمیز

این راهنما توضیح می‌دهد که این بسته چیست، چه کاری روی فایل شما انجام شد، داخل آن چه چیزهایی هست، و چطور قدم به قدم از آن استفاده کنید. برای خواندن آن نیازی به دانش فنی عمیق ندارید.

## ۱ این بسته چیست؟

تصور کنید یک «شرکت نرم‌افزاری کامل» دارید که همه اعضای عامل‌های هوش مصنوعی هستند: یک مدیر محصول، یک مدیر ارشد فنی، مهندس‌های بک‌اند و فرانت‌اند، تیم تست، تیم امنیت و غیره. این بسته دقیقاً همان چیز است – نه فقط چند «دستور» جدا از هم، بلکه یک **سازمان به هم پیوسته** با قانون اساسی، مسیرهای روشن تحویل کار، دروازه‌های کنترل کیفیت، و جاهایی که حتماً باید یک انسان تأیید کند.

این سیستم مستقل از زبان و فناوری است؛ یعنی برای هر پروژه‌ای (وب، موبایل، داده/هوش مصنوعی، زیرساخت و...) قابل استفاده است. نام رسمی آن **Virtual Software Organization** یا به اختصار **VSO** است.

**این بسته از کجا آمده؟** این چارچوب «مبتنی بر شواهد» است: از تحلیل یک مخزن واقعی به نام **credit-intelligence** (یک موتور اعتبارسنجی هوشمند برای بازار مالی ایران) استخراج شده. آن تحلیل اصلی به عنوان «مدرک فقط خواندنی» داخل همین بسته نگه داشته شده تا همیشه بدانید چه چیزی «اثبات شده» و چه چیزی «پیشنهادی» است.

## ۲ چه کاری روی فایل شما انجام دادم؟

فایل اصلی شما (**finalMultimanger.zip**) شلوغ بود: دو فایل زیپ تودرتو، ۱۷ فایل تکراری، و ۱۳۲ فایل زائد سیستمی داشت. همه را مرتب کردم:

### قبل (به هم ریخته)

زیپ داخل زیپ داخل زیپ

۱۳۲ فایل زائد سیستمی

۱۷ فایل کاملاً تکراری

ساختار گیج کننده

### بعد (تمیز)

یک پوشه واحد و منظم

• فایل زائد

• فایل تکراری

۱۹۶ فایل واقعی و مفید

### کارهایی که دقیقاً انجام شد:

- هر دو زیپ تودرتو را باز کردم و همه را در یک درخت پوشه واحد کنار هم چیدم.
- ۱۳۲ فایل زائد سیستمی مک ( \*\_ ، \_MACOSX ، .DS\_Store ) را حذف کردم.
- ۱۷ فایل .md که بایت به بایت کپی فایل های داخل پوشه blueprint/ بودند حذف شدند (نسخه اصلی سر جایش ماند).
- تحلیل اصلی (مدرک، ۱۰۴ فایل) را دقیقاً همان جایی گذاشتم که خود چارچوب می گوید باید باشد: `projects/credit-intelligence/source-extraction/original/`.
- هیچ فایل مفیدی حذف یا تغییر داده نشد؛ فقط زوائد و تکراری ها پاک شدند و بقیه مرتب شد.

چرا دو لایه جدا نگه داشته شد؟ پوشه اصلی ( `virtual-software-organization` ) همان چیزی است که استفاده می کنید. پوشه `source-extraction/original` فقط مدرک و مرجع است – نسخه خام و قدیمی تری که چارچوب از روی آن ساخته شده. اینها تکراری نیستند؛ دو نقش متفاوت اند و طبق قوانین همین سیستم باید جدا بمانند.

## ۳ داخل بسته چه چیزی هست؟

```
virtual-software-organization/
├── README.md      ← از اینجا شروع کنید
├── USAGE.md       ← راهنمای گام به گام استفاده
├── COMBINED-REFERENCE.md ← همه مستندات در یک فایل واحد
├── project-profile.example.yaml
├── framework/     ← سند از قوانین اصلی و بازاستفاده پذیر ۱۸
├── schemas/       ← قالب ماشین خوان (پیام، وظیفه، تحویل و...) ۱۲
├── templates/     ← قالب آماده پُرکردنی ۱۵
├── diagrams/      ← نمودار (Mermaid) ۱۰
├── projects/credit-intelligence/ ← یک نمونه کامل کار شده
│   ├── audit/     ← بازبینی کیفیت تحلیل
│   └── blueprint/ ← چارچوب اجرا شده روی نمونه
```

| پوشه         | کاربرد به زبان ساده                                                     |
|--------------|-------------------------------------------------------------------------|
| framework/   | مغز سیستم؛ قوانین کلی که برای همه پروژه‌ها یکسان است.                   |
| schemas/     | ساختار استاندارد داده‌ها تا همه عامل‌ها یک‌زبان صحبت کنند.              |
| templates/   | فرم‌های آماده برای پیام، تحویل کار، گزارش تست، درخواست تأیید و...       |
| diagrams/    | تصویر کلی سازمان، جریان کار و دروازه‌های کیفیت.                         |
| projects/... | یک مثال واقعی کامل که نشان می‌دهد همه‌چیز در عمل چطور کنار هم می‌نشیند. |

## ۴ سه فایلی که تحویل می‌گیرید

- ۱ **virtual-software-organization-CLEAN.zip** – کل سیستم تمیز و مرتب در یک فایل. این همان چیزی است که باز می‌کنید و استفاده می‌کنید.
- ۲ **COMBINED-REFERENCE.md** – همه مستندات مهم (معرفی، ۱۸ سند قوانین، ۱۰ نمودار و نمونه کارشده) در «یک فایل واحد خواندنی». اگر نمی‌خواهید ده‌ها فایل را جدا باز کنید، این را بخوانید. یک نسخه از آن داخل زیپ هم هست.
- ۳ **همین فایل راهنمای PDF** – توضیح ساده و کاربردی که الان در حال خواندنش هستید.

## ۵ چطور استفاده کنم؟ (گام‌به‌گام)

- ۱ **بخوانید.** اول **README.md** و بعد **USAGE.md** را باز کنید تا تصویر کلی دستتان بیاید.
- ۲ **پروژه بسازید.** فایل **project-profile.example.yaml** را کپی کنید در **projects/<نام‌پروژه>/project-profile.yaml** و یک «پروفایل» انتخاب کنید (جدول بخش ۶).
- ۳ **زمینه را پُر کنید.** از روی **PRD** خود (سند نیازمندی‌ها)، فایل **state/shared-context.yaml** را تکمیل کنید: هدف‌ها، دامنه، قوانین کاری، معیارهای پذیرش و...
- ۴ **وضعیت را آماده کنید.** فایل‌های **state/project-state.yaml** و دفترهای تصمیم/ریسک/تأیید را بسازید تا کار پس از هر وقفه از همان‌جا ادامه یابد.
- ۵ **به Claude Code بسپارید.** فایل **blueprint/claude-code-handoff-prompt.md** را به Claude Code بدهید تا عامل‌ها، قلاب‌ها و ابزارها را روی یک شاخه جداگانه بسازد – بدون دست‌زدن به کد برنامه.
- ۶ **جریان کار را اجرا کنید.** از **framework/11-prd-to-production-workflow.md** پیروی کنید: از PRD تا استقرار، مرحله‌به‌مرحله با تأییدهای لازم.

## ۶ پروفایل پروژه (اندازه سازمان را تعیین می‌کند)

لازم نیست یک پروژه کوچک همان سازمان بزرگ یک پروژه پرخطر را داشته باشد. با انتخاب «پروفایل»، مشخص می‌شود کدام عامل‌ها، دروازه‌ها و تأییدهای انسانی فعال شوند:

| پروفایل                               | چه زمانی؟                                                   |
|---------------------------------------|-------------------------------------------------------------|
| minimal                               | ابزار کوچک، کم‌خطر، یک صفحه.                                |
| standard                              | محصول معمولی با رابط کاربری و API.                          |
| high_risk                             | پول، ایمنی، کارهای برگشت‌ناپذیر – همه تأییدهای انسانی روشن. |
| regulated                             | تعهدات قانونی/انطباقی؛ امنیت و مستندسازی بیشتر.             |
| data_or_ai                            | خطوط داده و مدل‌های یادگیری ماشین.                          |
| mobile / backend_only / frontend_only | پروژه‌های تک‌تخصصی.                                         |

## ۷ قوانین طلایی سیستم

- هر تصمیم دقیقاً یک صاحب دارد.
- کسی که کار را انجام می‌دهد، نمی‌تواند کار نهایی خودش را تأیید کند (بازبین مستقل لازم است).
- هماهنگ‌کننده (orchestrator) فقط هماهنگ می‌کند و اختیار تصمیم ندارد.
- مهندس امنیت یک «وتو محدود» دارد که حتی مدیر فنی هم نمی‌تواند لغوش کند.
- یک کار فقط وقتی «تمام» است که مدرک عبور از دروازه‌های کیفیت را داشته باشد.
- کارهای پرخطر حتماً تأیید انسانی می‌خواهند.
- کار نیمه‌تمام هیچ‌وقت بی‌سروصدا «ترمیم» نمی‌شود؛ برمی‌گردد به صاحبش.

## ۸ سؤال‌های پرتکرار

آیا چیزی از فایل من پاک شد که مهم باشد؟

نه. فقط فایل‌های زائد سیستمی و نسخه‌های کاملاً تکراری حذف شدند. تمام محتوای اصلی – از جمله تحلیل خام – دست‌نخورده داخل بسته است.

فرق پوشه اصلی با `source-extraction/original` چیست؟

پوشه اصلی «چارچوب آماده برای استفاده» است. آن یکی «مدرک و بایگانی» است تا همیشه بشود ردیابی کرد هر تصمیم از کجا آمده. لطفاً مدرک را ویرایش نکنید.

## برای شروع باید حتماً برنامه‌نویس باشم؟

برای «استفاده» نه، کافی است `README.md` و `USAGE.md` را بخوانید و پروفایل و زمینه را پُر کنید. ساختِ بخشِ اجرایی را می‌توانید به Claude Code بسپارید.

## چطور همین سیستم را برای چند پروژه به کار ببرم؟

برای هر پروژه فقط یک پوشهٔ تازه زیرِ `projects/<نام>` بسازید. پوشه‌های `framework/`، `schemas/` و `templates/` مشترک و بدون تغییر می‌مانند — آنها را برای هر پروژه کپی نکنید.

---

ساخته‌شده برای تیمِ محصولِ صراف · این راهنما فقط توضیح و آموزش است و کد هیچ برنامه‌ای را تغییر نداده است. برای جزئیاتِ کامل، به `COMBINED-REFERENCE.md` یا فایل‌های داخلِ زیپ مراجعه کنید.